

Building a **Digital Cognitive Architecture**

A complete step-by-step guide to building a persistent cognitive substrate for intelligent systems that perceive, remember, reason, plan, and adapt

From Idea → Research → Prototype → Autonomous System

What Is a Cognitive Architecture?

Before writing a single line of code, you need to understand what you are actually building — and more importantly, what you are not building.

Most people who encounter this idea immediately anchor it to a specific use case: "a microservices health checker," "an AI DevOps tool," "a monitoring dashboard with AI." That framing is dangerously narrow. A cognitive architecture is not a product category. It is a structural paradigm — a way of organizing intelligence — that can be applied to virtually any domain where an entity needs to perceive, understand, remember, reason, plan, and act in a changing environment.

The Universal Nature of Cognitive Architecture

Consider what all of the following have in common:

- A robot navigating a warehouse, picking and placing items
- A financial trading system that reads market signals and manages a portfolio
- A game AI that plays StarCraft or chess at a superhuman level
- A scientific research assistant that reads papers, forms hypotheses, and designs experiments
- An autonomous vehicle managing traffic, pedestrians, and route decisions
- A personal AI companion that remembers your history and adapts to your needs over years
- An infrastructure orchestration system that manages cloud resources across data centers
- A medical diagnostic system that builds a patient model and reasons over symptoms over time

All of these require the same fundamental capabilities: the ability to perceive an environment, build a model of it, remember what happened, reason about what is happening, plan what to do, execute actions, and update the model based on results. The domain changes. The underlying cognitive structure does not.

The Core Principle

A cognitive architecture is a general framework for producing intelligent behavior in any environment. The components — perception, memory, world model, reasoning, planning, execution, and learning — are domain-agnostic. You build the architecture once; you wire it to different environments to get different intelligent systems.

Why Most AI Systems Are Not Cognitive Architectures

The current landscape of "AI systems" is dominated by what we might call model-centric thinking: take an LLM, wrap it in an API, call it an agent, ship it. These systems have fundamental structural limitations:

Limitation	What It Means	Why It Matters
Stateless	Each prompt forgets the last	No continuity, no genuine learning from interaction
Ungrounded	No connection to real-world data unless manually injected	Hallucination when asked about current state
No world model	Cannot represent how things relate or evolve	Cannot reason causally about what will happen
No memory architecture	RAG is not memory — it is text retrieval	Cannot build up experience over time like a person does
Reactive only	Responds to inputs; cannot pursue goals proactively	Cannot plan ahead or self-correct without prompting
No feedback loop	Actions are not observed for consequences	Cannot improve its own behavior based on outcomes

A true cognitive architecture addresses every one of these limitations by design.

The Seven Pillars of a Cognitive Architecture

Every genuine cognitive architecture, regardless of domain, is built on these seven capabilities. Think of them as organs: a system is only cognitively complete when all seven are present and connected.

Pillar	What It Provides
Environment Awareness	The ability to perceive the current state of the environment in real time, across all relevant input channels
Memory	The ability to store, organize, and retrieve different types of information over different time horizons
World Modeling	A structured, queryable representation of what exists, how things relate, and how they change over time
Reasoning	The ability to analyze situations, evaluate options, understand causality, and explain decisions
Planning	The ability to decompose goals into executable subgoals, sequences, and contingency strategies
Execution	The ability to translate plans into concrete actions that affect the environment
Learning	The ability to update internal models and strategies based on the outcomes of past actions

These are not sequential steps — they are simultaneous, continuously interacting processes. Perception feeds memory. Memory feeds the world model. The world model feeds reasoning. Reasoning feeds planning. Planning feeds execution. Execution feeds back into perception. This closed loop is what creates autonomous, adaptive behavior.

Why We Are Starting with Digital Cognitive Architecture

The same cognitive structure that makes a robot navigate a room can make a software system manage a data center, or help a research assistant track hundreds of scientific papers, or enable an autonomous trading system. We start in the digital domain not because the physical domain is unimportant, but because:

- There is no hardware cost or complexity — iteration is cheap and fast
- The environment is perfectly observable — every signal is already digital
- Mistakes are reversible — a bad action in software does not break a robot arm
- The architecture transfers directly — every component you build digitally can later interface with physical sensors and actuators

The monitoring of services and infrastructure is our first prototype domain — not our final vision. It is chosen because it makes every cognitive capability immediately testable and demonstrable. Real signals. Real failures. Real decisions. Real consequences.

The Complete Architecture: How Everything Connects

Before diving into phases, you need a complete mental picture of the system you are building. Every component connects to every other component. Understanding these connections prevents you from building components in isolation that later refuse to integrate.

The Layered Architecture

The system has five functional layers. Think of them vertically: the bottom layer faces the environment; the top layer produces actions that affect the environment. Data flows up; actions flow down; feedback connects the bottom to the top.

Layer	Function
Layer 1: Perception	Ingests raw signals from the environment. Normalizes them into a common event schema. This is the sensory system.
Layer 2: Memory	Routes normalized events into the right memory store. Maintains four distinct memory types with different time horizons and retrieval mechanisms.
Layer 3: World Model	Synthesizes memory into a coherent, queryable, evolving representation of the environment. This is what the system "knows" about reality right now.
Layer 4: Reasoning & Planning	Queries the world model to understand situations, evaluate options, and construct executable plans. LLMs are one reasoning component here.
Layer 5: Execution	Translates plans into concrete actions. Every action produces environmental change, which generates new perception events, completing the loop.

The Critical Connections Between Layers

The connections between layers are as important as the layers themselves. Here is exactly how they interact:

Perception → Memory Router

Every normalized event produced by the perception layer passes through a memory router. The router does not store data itself — it classifies each event and dispatches it to the correct memory store. An error event goes to episodic memory. A new service discovery goes to the knowledge graph. An active task update goes to working memory. A deployment procedure that was successfully executed goes to procedural memory. This routing decision happens in real time, on every event, before anything else downstream sees the data.

Memory → World Model

The world model does not store data — it reads from memory to construct a live view of reality. It queries the knowledge graph for current relationships between entities, reads recent events from episodic memory to understand trends, checks working memory for the current goal and task context, and reads procedural memory when it needs to know how something is typically done. The world model is a synthesizer: its job is to answer the question "what is the state of the environment right now, in structured form?"

World Model → Situation Assessor

The raw world model output can be very large — hundreds of entities, thousands of relationships, long event histories. Before passing it to a reasoning engine, a situation assessor compresses it into a structured situation summary: what is the most important thing happening right now, what is its severity, what entities are involved, what has already been tried. This compression step is what makes LLM-based reasoning reliable and cheap.

Situation Assessor → Reasoning Engine

The reasoning engine receives a compact, structured situation description and is responsible for producing a decision: what should be done next, and why. The reasoning engine can use an LLM, a rule-based system, a trained policy model, or any combination. The key architectural insight is that the reasoning engine is one component inside the architecture — not the architecture itself. It does not perceive. It does not remember. It does not execute. It only reasons over what it is given.

Reasoning Engine → Planner

The reasoning engine produces a decision — "scale the database read replicas." The planner turns that decision into an executable sequence: check current replica count, determine target count, call scaling API, wait for readiness, verify latency improvement, update the world model. The planner handles sequencing, dependencies, failure conditions, and rollback strategies. It produces a structured plan that the execution layer can follow step by step.

Execution → Feedback

Every action the execution layer takes changes the environment. That change produces new signals. Those signals flow through perception, update memory and the world model, and the reasoning engine re-evaluates the situation. This feedback loop is what creates adaptation: the system acts, observes the result, and adjusts its next action accordingly. A system without a properly implemented feedback loop is a pipeline, not a cognitive architecture.

The Event Bus: The Connective Tissue

All of these layers communicate through a central event bus — a persistent, ordered stream of events that every layer can both read from and write to. This design provides several critical benefits:

- Decoupling: layers do not call each other directly; they communicate through events, so each layer can be developed and tested independently
- Replay: you can replay past events to test whether the system would behave differently with an improved world model or reasoning engine
- Observability: every event that passes through the bus is automatically traceable — you can reconstruct any decision the system ever made
- Multi-agent support: multiple agents can subscribe to the same event stream without any architectural changes

The Key Mental Model

Think of the event bus as the bloodstream of the cognitive architecture. Perception produces blood cells (events). Memory stores them in different organs (stores). The world model is the brain's live picture of what the body is doing. Reasoning and planning are conscious thought. Execution is motor action. Feedback is the sensory signal that tells the brain what the action

Phase 1

Build the Mental Model

Study before you code — understand what you are building and why

This is the most important phase, and it requires no code. The time you spend here determines whether the system you build has a coherent architecture or becomes a pile of disconnected scripts. Every hour of study in Phase 1 saves ten hours of refactoring in Phase 4.

What We Are Doing

We are studying five bodies of knowledge that directly inform how the cognitive architecture is structured. Each one maps to a specific layer or capability in the system.

Why We Are Doing This

Most developers skip this phase and start building immediately. The result is architectures that look reasonable on day one and collapse under complexity by week three. The cognitive architecture pattern is not obvious — it runs counter to the typical API-wrapper style of AI development. Without understanding why each component exists and how it connects to the others, you will repeatedly make design decisions that seem sensible locally but create integration problems globally.

What to Study and How

A. Cognitive Architectures (SOAR, ACT-R, Global Workspace Theory)

These are decades-old computational models of how human cognition is organized. You do not need to implement them — you need to understand the structural decisions they embody.

What to Learn

SOAR: how a system with working memory, long-term memory, and a problem space can pursue goals. ACT-R: the distinction between procedural memory (how to do things) and declarative memory (facts about the world). Global Workspace Theory: why a shared "blackboard" that all modules read from creates emergent intelligent behavior.

How to study: Read the Wikipedia articles first for orientation. Then read "How Can the Human Mind Occur in the Physical Universe?" by John Anderson (ACT-R) for the memory architecture insight. For SOAR, read the introductory chapters of "The Soar Cognitive Architecture." Do not go deep on implementation — extract the structural ideas.

B. World Models and Latent State Modeling

A world model is an internal representation of the environment that the system uses for reasoning rather than querying the environment directly. This is the concept that elevates a reactive system into a predictive one.

What to Learn

Read Yann LeCun's "A Path Towards Autonomous Machine Intelligence" paper — it contains the clearest explanation of why world models are the missing piece in current AI systems. Study scene graphs as a data structure: how objects and relationships are represented as a graph where nodes are entities and edges are relationships. Understand the difference between observed state (what sensors report) and latent state (what the system infers must be true given

C. Knowledge Graphs and Graph Databases

The knowledge graph is the structural core of the world model. Before building one, you need to understand what graphs are good at and what they are not.

What to Learn

Study the property graph model: nodes have labels and properties; edges have types and properties. Learn Cypher query language (used in Neo4j) — specifically how to express questions like "which services depend on this database?" as graph traversals. Understand the difference between a relational database (good at tabular queries) and a graph database (good at relationship traversals of arbitrary depth).

D. Event-Driven Systems and Temporal Databases

Cognitive architecture is inherently temporal. Things change. The system must track not just what is true now but what was true before, and use that history to predict what will be true next.

What to Learn

Understand event sourcing: rather than storing current state, store the sequence of events that produced the current state. This gives you full history for free. Study Kafka's log-based architecture: partitions, consumer groups, offsets. Learn about time-series databases and why they are different from relational databases for temporal queries. Study sliding window aggregations: how to compute "average error rate over the last five minutes" as a streaming operation.

E. Multi-Agent Systems

Even if you start with a single agent, the architecture should be designed from day one to support multiple agents. Multi-agent design constraints produce better single-agent architectures because they force you to be explicit about what is shared state versus local state.

What to Learn

Study the actor model: agents are independent entities that communicate only through message passing. Read about shared blackboard architectures. Study LangGraph specifically — it represents agent workflows as directed graphs, which maps directly onto the planning layer of a cognitive architecture. Understand the distinction between a planner agent (decides what to do), an executor agent (does it), and a monitor agent (verifies it was done correctly).

Deliverable for Phase 1

Before moving to Phase 2, you should be able to answer these questions confidently:

1. What is the difference between working memory and episodic memory, and why does the distinction matter?
2. Why is a knowledge graph better than a relational database for representing service dependencies?
3. What is event sourcing and why does it give you temporal cognition for free?
4. Why should the LLM be a component inside the architecture rather than the architecture itself?
5. What is the feedback loop and what would break if it were missing?

Phase 2

Define the Unified Event Schema

The contract that all layers speak — define this before writing any other code

This is the single most impactful design decision you will make in the entire project. The event schema is the contract between every component in the system. If the schema is well-designed, integration between layers is straightforward. If it is poorly designed, you will spend weeks retrofitting every component every time you add a new data source.

What We Are Doing

We are defining the universal data structure that every event in the system — regardless of source — will conform to. This schema is the common language of the entire architecture. Perception produces events in this format. Memory consumes them. The world model queries them. The reasoning engine reads them. The feedback loop writes them.

Why Before Everything Else

Consider what happens without a unified schema: your Kafka consumer for logs produces one format, your API health checker produces another, your metrics pipeline produces a third. The world model has to understand three different formats. When you add a fourth source, you touch the world model again. The memory router has to be updated. The reasoning engine has to handle special cases. Every integration point multiplies the complexity.

With a unified schema, every new data source requires only one thing: a normalization adapter that converts its native format into the standard schema. Everything downstream is already compatible.

The Schema Design

Every event in the system must carry this envelope:

Field	Type	Purpose
<code>event_id</code>	UUID	Globally unique identifier — enables deduplication and tracing across the entire system
<code>timestamp</code>	ISO 8601	When the event actually occurred in the environment — not when it was processed
<code>ingested_at</code>	ISO 8601	When the perception layer received it — the gap between this and timestamp reveals lag
<code>source_type</code>	Enum	Category of source: log, metric, api, queue, file, user_action, agent_action, external_service

<code>source_id</code>	String	Specific identifier of the source: "auth_service_pod_3", "payment_api", "user_42"
<code>entity_id</code>	String	The primary entity this event is about — maps to a node in the knowledge graph
<code>entity_type</code>	Enum	What kind of entity: service, user, task, file, database, agent, device, environment
<code>event_type</code>	String	What happened: "cpu_spike", "request_failed", "file_created", "goal_completed", "anomaly_detected"
<code>severity</code>	Enum	critical, high, medium, low, info — for prioritization in the reasoning engine
<code>payload</code>	JSON	Domain-specific data for this event type — unstructured but validated against a type-specific schema
<code>confidence</code>	Float 0-1	How certain the perception layer is about this event — accounts for noisy sensors and inference
<code>causal_event_id</code>	UUID?	If this event was caused by another event, reference it here — enables causal chain tracing
<code>tags</code>	String[]	Arbitrary labels for filtering and routing: ["production", "database", "cpu", "escalating"]

The Confidence Field: Why It Matters

Every event carries a confidence score between 0 and 1. This is not optional. Here is why: sensors lie, logs are incomplete, inferred events are uncertain. A system that acts without representing its own uncertainty will make confident wrong decisions. The confidence field forces you to think about uncertainty at ingestion time — before it propagates silently through memory and world model into a bad plan.

A log that says "ERROR 504" has confidence 1.0 — it definitely happened. An inference that says "this service is probably failing based on upstream latency" has confidence 0.7. The reasoning engine treats these differently.

How to Implement

Define this schema as a Pydantic model (Python) or Zod schema (TypeScript). Every data adapter — your Kafka consumer, your API health checker, your log parser — is required to produce events that validate against this schema before they are accepted by the event bus. Reject events that do not conform. This enforcement is what keeps the downstream system clean.

Technology Choice

Use Python with Pydantic v2 for schema definition and validation. It gives you free JSON serialization, automatic validation with clear error messages, and generates JSON Schema for documentation. Define the schema in a shared library imported by every component.

Phase 3

Build the Perception Layer

The sensory system — continuously ingesting and normalizing environmental signals

The perception layer is the system's contact with reality. Everything the cognitive architecture knows about the world comes through this layer. Its job is simple in concept but requires careful implementation: take raw, heterogeneous signals from the environment and produce clean, normalized events in the unified schema.

What We Are Doing

We are building a pipeline of adapters — one per signal source — each of which ingests raw data, normalizes it into the unified event schema, attaches confidence scores, and publishes it to the central event bus. We are also building the event bus itself.

What Goes Into the Perception Layer

In the digital domain, your environment is everything that can be observed or measured. The perception layer is not limited to infrastructure monitoring — it can ingest anything:

Domain	Signal Sources	Example Events
Infrastructure	Prometheus, logs, APM tools	CPU spike, memory leak, service restart
User behavior	Clickstream, session logs, auth events	Login failure, feature usage, abandonment
Business processes	Workflow systems, CRM, queues	Order placed, payment declined, SLA breach
Data pipelines	Kafka topics, file watchers, database changes	Schema change, pipeline lag, data quality alert
External systems	Third-party APIs, webhooks, feeds	Vendor outage, rate limit hit, price change
Agent actions	Execution layer output	Action completed, plan step failed, goal achieved
Research / knowledge	RSS feeds, paper APIs, web scraping	New paper published, citation updated, concept added
Physical sensors	IoT, GPS, cameras (via adapters)	Temperature threshold, location update, object detected

Critical Point

The perception layer is domain-agnostic by design. You are not building a "monitoring tool." You are building a universal signal ingestion and normalization system. The domain is determined by what adapters you write, not by the architecture itself. This is why the same perception layer that monitors services today can track user behavior tomorrow and read scientific papers the day after.

The Architecture of the Perception Layer

Each data source has a dedicated adapter. The adapter has three responsibilities:

6. Ingest: connect to the source and receive raw data in whatever format the source produces
7. Normalize: extract the relevant fields and map them to the unified event schema
8. Publish: write the validated event to the event bus

The adapters are stateless. They do not store data, make decisions, or interact with any layer other than the event bus. This simplicity is deliberate — it makes each adapter independently testable and replaceable.

The Event Bus

Use Apache Kafka as the event bus. This is not the only option, but it is the right choice for a cognitive architecture because:

- Events are persisted on disk — they are not lost if a consumer is down
- Multiple consumers can read the same stream independently — memory, world model, and observability can all consume the same events without coordination
- Events can be replayed from any point in history — essential for debugging and for testing improved reasoning models against past events
- Kafka scales horizontally without any architectural changes

Normalizer Design Pattern

Every adapter follows this pattern: raw input arrives; a parser extracts structured fields; a mapper translates those fields to the event schema; a validator confirms the result; and the event is published. Crucially, the normalizer must handle ambiguity gracefully — if a field is missing or uncertain, it sets the confidence score appropriately rather than failing or silently dropping data.

What to Build First

Start with exactly three adapters: a log file adapter that reads application logs and parses common error patterns, a metrics adapter that reads from Prometheus or a similar source, and a synthetic event generator that produces test events in the correct schema format. The synthetic generator is not throwaway code — it becomes your test harness for every downstream component.

Technology Stack for Phase 3

Component	Technology
Event bus	Apache Kafka (or Redpanda for development — same API, easier to run locally)
Adapter framework	Python with asyncio for concurrent adapter execution
Schema validation	Pydantic v2 — validates every event before it touches the bus
Log parsing	Loguru for structured logging; custom parsers for specific log formats
Metrics ingestion	prometheus_client or direct HTTP to Prometheus API
Kafka client	confluent-kafka-python — most reliable Python Kafka client
Testing	pytest with a synthetic event generator; validate that every adapter produces schema-conformant events

Phase 4

Build the Memory Systems

Four distinct memory types — each with a different time horizon and retrieval mechanism

Memory is where most AI systems fail. They either have no memory (stateless LLM wrappers), or one undifferentiated memory store (vector databases containing everything), or memory that is never actually used in reasoning. A cognitive architecture requires four distinct types of memory because different kinds of information have different retrieval patterns, different expiry times, and different roles in the reasoning process.

The Four Memory Types

Working Memory — The Active Context

Working memory holds the current state of whatever the system is actively engaged in. It is fast, small, and temporary. Think of it as the system's "current thought": what goal is being pursued, what task is in progress, what were the last five relevant events, what hypotheses are currently being evaluated.

Working memory is not a database — it is a live state object. It expires automatically. If the system crashes and restarts, working memory is rebuilt from the event bus history (a critical reason for Kafka's persistence).

Technology

Redis with a TTL (time-to-live) on every key. Use Redis Streams for the "last N events" buffer. Typical TTL: seconds to minutes depending on the use case.

Episodic Memory — The Experience Store

Episodic memory stores the history of what happened, when, and what resulted. It is the source of experiential learning. When the reasoning engine asks "have we seen this situation before?", it queries episodic memory. When the world model needs to compute trends, it queries episodic memory.

Episodic memory is implemented in two complementary stores: a time-series database for metrics and temporal queries (give me all events involving the auth service in the last hour), and a vector database for semantic similarity queries (find past situations similar to this current one).

Technology

PostgreSQL with TimescaleDB extension for time-series queries. Qdrant for vector similarity search (embed each event using a small embedding model so you can find semantically similar past events). The combination gives you both exact temporal queries and fuzzy semantic recall.

Semantic Memory — The Knowledge Store

Semantic memory stores persistent facts about the world: what entities exist, how they relate, what their properties are, what their history looks like at a structural level. This is the knowledge graph. It is the most important memory store for the world model.

Unlike episodic memory (a log of events), semantic memory represents stable knowledge: "Service A depends on Database B." This relationship is not an event — it is a fact that is true until something changes it. The knowledge graph stores facts; the event stream stores changes to facts.

Technology

Neo4j for the property graph database. Use the Cypher query language. Model entities as nodes with labels (Service, Database, User, Task, Agent) and relationships as edges with types (DEPENDS_ON, OWNED_BY, TRIGGERS, COMMUNICATES_WITH).

Procedural Memory — The How-To Store

Procedural memory stores knowledge about how to do things: what steps are involved in deploying a service, what the rollback procedure is for a failed migration, what the standard response to a database timeout looks like. This is distinct from episodic memory (which records what was done) and semantic memory (which records what exists).

Procedural memory is consulted by the planner: before constructing a new plan, the planner checks whether a procedure for this type of situation already exists. If it does, the planner instantiates it rather than reasoning from scratch. Over time, successful plans are stored back into procedural memory, making the planner faster and more reliable.

Technology

PostgreSQL with a structured procedures table. Each procedure has a trigger condition (expressed as a world state predicate), a sequence of steps, preconditions, postconditions, and a success rate tracking column that is updated by the feedback loop.

The Memory Router

The memory router is the critical component that most architecture documents omit. It sits between the event bus and all four memory stores. It subscribes to every event on the bus and makes four decisions: does this event update working memory? Does it go to episodic memory? Does it trigger a knowledge graph update? Does it update or create a procedure?

The routing rules are not arbitrary — they follow from the nature of each memory type. Every event goes to episodic memory — history is always recorded. Events that assert or update entity relationships go to the knowledge graph. Events that represent the current active task update working memory. Events that represent the successful completion of a planned sequence update procedural memory.

Why Four Memory Types and Not One

You might wonder: why not just put everything in one vector database and let the retrieval model figure out what is relevant? Here is the concrete answer:

- Vector search is semantic but not temporal — it cannot answer "what happened in the last five minutes?" efficiently
- Graph databases are relational but not temporal — they answer "how are these things connected?" not "what just happened?"
- Time-series databases are temporal but not relational — they answer time-window queries but cannot traverse relationships
- Working memory is none of the above — it is a live state object that needs millisecond access, not a database at all

Each store is the right tool for specific queries. The world model uses all four — they are complementary, not alternatives.

Phase 5

Build the Knowledge Graph

The structural backbone of the world model — what exists and how it relates

The knowledge graph is the system's structured understanding of the world. It represents not raw events, but knowledge: the stable facts and relationships that make sense of those events. This is the difference between "error event received from auth_service at 14:32" (episodic memory) and "auth_service depends on postgres_primary, which is currently in degraded state" (knowledge graph).

What the Knowledge Graph Stores

The graph stores entities as nodes and relationships as edges. Both nodes and edges can carry properties.

Node Types (Entities)

Design your node labels to reflect your domain. For a general cognitive architecture, start with these universal types and extend as needed:

Node Label	What It Represents
Service	Any running process, microservice, or application component
Database	Any data store: SQL, NoSQL, cache, queue, file system
Agent	Any cognitive agent in the system, including sub-agents
User	Any human actor interacting with the system
Task	Any unit of work being tracked: goal, subtask, job, procedure
Resource	Any infrastructure resource: server, container, network, storage
Concept	Any abstract knowledge entity: a domain concept, a known failure mode, a strategy
Environment	Any external system or context: a cloud region, an organization, a physical location

Relationship Types (Edges)

Relationship Type	Meaning
DEPENDS_ON	A requires B to function — a failure in B affects A
COMMUNICATES_WITH	A sends requests to B — important for tracing request flows
OWNED_BY	A is managed or owned by B (user, team, or agent)

TRIGGERS	When A happens, B is initiated or scheduled
CONTAINS	A is a component of B — for hierarchical containment
CAUSED_BY	This event or state was caused by another event or state — causal tracing
MONITORS	Agent A is responsible for watching entity B
PREVIOUSLY_RESOLVED_BY	This type of problem was previously fixed by this procedure

How the Knowledge Graph Is Updated

The memory router updates the knowledge graph when events assert new facts or change existing ones. There are three types of graph updates:

9. Node property updates: an event changes a property on an existing node. A service's "status" property changes from "healthy" to "degraded." This is a simple MERGE operation in Cypher.
10. New relationships: an event reveals a relationship that was not previously known. A service starts calling a new API endpoint. A new dependency edge is created.
11. Node creation: a new entity appears in the environment. A new service is deployed. A new agent is spawned. A new user registers.

The graph is also updated by the reasoning layer: when the reasoning engine determines a causal relationship ("the queue buildup was caused by the database slowdown"), it writes a CAUSED_BY edge back into the graph. This is how the system accumulates causal knowledge over time.

Why the Graph Enables Reasoning That Flat Databases Cannot

Consider a question like: "Which users are likely to be affected by the current database outage?" A relational database requires you to know the join path in advance. A graph database answers this by traversal: start at the degraded database node, follow DEPENDS_ON edges backward to find services, follow COMMUNICATES_WITH edges to find entry-point services, follow OWNED_BY or session data to reach users. The answer emerges from the structure of the graph itself — you do not need to know the query in advance, only the direction of traversal.

This is the power of graph-based world modeling: the intelligence is in the structure, not just in the data. Causal chains, dependency paths, and impact radii are all native queries, not post-hoc analysis.

Phase 6

Build Temporal Awareness

How state evolves over time — making the system cognizant of past, present, and predicted future

A system that only knows the current state of the world is reactive. A system that knows how the current state was reached, how quickly things are changing, and what patterns typically precede failures is predictive. Temporal awareness is what separates a monitoring dashboard from a cognitive agent.

What Temporal Awareness Means

The system must maintain three temporal perspectives simultaneously:

- The past: what happened, in what sequence, with what outcomes — stored in episodic memory
- The present: what is the current state of every relevant entity — stored in the knowledge graph and working memory
- The predicted future: given current trends and historical patterns, what is likely to happen next — computed by the temporal analysis layer

Temporal Analysis Components

Event Stream Processing

Use Apache Flink or a simpler streaming library to compute continuous aggregations over the event stream. Examples: rolling average error rate per service over the last ten minutes; rate of change of queue depth; count of failed requests in the last sixty seconds. These aggregations are published back to the event bus as derived events — they go through the same normalization and memory routing as any other event.

Trend Detection

Beyond simple aggregations, the temporal layer computes trends: is this metric increasing, decreasing, or stable? Is the rate of change accelerating? Implement this with simple statistical methods first — linear regression over a sliding window to compute slope, standard deviation for anomaly detection. Replace with ML models only after the simpler version is running reliably.

Pattern Matching

Some events only become meaningful in the context of a sequence. A single database timeout is noise. Three database timeouts in five minutes followed by increasing queue depth is a pattern — one that historically precedes a service outage. Pattern matching

over the event stream turns individual observations into higher-level situational awareness.

Implement this with complex event processing (CEP): define patterns as sequences or combinations of event types within time windows, and emit a new higher-confidence derived event when the pattern is matched.

State Versioning in the Knowledge Graph

The knowledge graph represents the current state of the world. But sometimes you need to know what the state was at a specific past time — especially during incident analysis. Implement temporal versioning on graph nodes: every property change records the previous value, who changed it, and when. This enables time-travel queries: "what was the state of this service five minutes before the outage?"

The Temporal Confidence Update

Confidence scores on events decay over time. An observation made ten minutes ago is less reliable than one made ten seconds ago — the environment may have changed in the interim. The temporal layer is responsible for emitting "staleness events" that signal to the world model that certain observations need to be refreshed. This prevents the system from reasoning on stale data with full confidence.

Phase 7

Build the World Model

The unified, queryable representation of everything the system knows about its environment

The world model is not a database, a graph, or a time-series store. It is a synthesis layer — a live, queryable view of reality constructed by continuously reading from all four memory stores and combining their outputs into a coherent picture of the current environment. This is the component that all reasoning and planning queries.

What the World Model Does

The world model answers four fundamental questions that no single memory store can answer alone:

- What exists? (entities and their current properties — from the knowledge graph)
- What is changing, and how fast? (trends from temporal analysis and episodic memory)
- Why did it change? (causal chains from the knowledge graph's CAUSED_BY edges)
- What is likely to happen next? (predictions from pattern matching and trend extrapolation)

The World Model as a Query Interface

The world model exposes a structured query API — not a natural language API, not a SQL API, but a purpose-built interface designed for the queries that reasoning engines and planning systems actually need to make. Examples:

- `get_entity_state(entity_id)`: returns the current properties of an entity along with recent events and confidence
- `get_dependencies(entity_id, depth)`: returns all entities that entity depends on, up to a given graph depth
- `get_causal_chain(event_id)`: traces backward through CAUSED_BY edges to find the root cause of an event
- `get_impact_radius(entity_id)`: finds all entities that would be affected if this entity fails
- `get_situation_summary()`: returns a compressed description of the most important current events, entities, and trends
- `get_similar_past_situations()`: queries episodic memory for past situations semantically similar to the current one

The Situation Assessor

The world model's raw output is too large and detailed for direct LLM consumption. The situation assessor is a lightweight component — implemented as a combination of rule-

based filters and a small ML classifier — that reads the full world model state and compresses it into a structured situation summary.

The situation summary contains: the current severity level, the primary entities involved, the most significant recent events (ordered by importance), the current causal hypothesis (if one exists), what has already been attempted, and what constraints apply (budget, risk tolerance, business hours). This is what the reasoning engine actually receives.

Why This Step Cannot Be Skipped

An LLM given a 10,000-token world model state will hallucinate, lose track of context, and miss critical details. An LLM given a 500-token structured situation summary focused on what matters right now will reason reliably. The situation assessor is the difference between an LLM that reasons well and one that fails unpredictably.

Dynamic Modeling: Predictions

Once sufficient episodic data has accumulated (typically several weeks of operation), the world model can begin making predictions. Start simple: linear trend extrapolation ("at the current growth rate, the queue will reach critical capacity in 4 minutes"). Graduate to more sophisticated models as the data matures: train a small gradient boosting model on historical event sequences to predict failure probabilities.

Do not use GNNs or deep learning at this stage. The episodic memory does not yet have enough data for these models to be meaningful, and the simpler models will serve the reasoning engine just as well while you build up the data foundation.

Phase 8

Build the Reasoning Engine

Where the system thinks — analyzing situations, evaluating options, explaining decisions

The reasoning engine is where the system moves from knowing to deciding. It receives a structured situation summary from the world model, consults its available options and constraints, and produces a decision: what should be done next, with what justification, and with what level of confidence.

The Critical Design Principle

LLMs Are a Component, Not the Architecture

This cannot be overstated. An LLM is a powerful reasoning component — it excels at evaluating ambiguous situations, generating creative options, explaining decisions in natural language, and handling novel scenarios. But it has no memory, no world model, no perception, no ability to act. In this architecture, the LLM receives a structured input and produces a structured output. It is one organ in the body — not the body itself.

The Reasoning Engine's Responsibilities

- Situation analysis: given the situation summary, what is actually happening? What is the root cause? What is the severity?
- Option generation: what are the possible responses to this situation? What are the tradeoffs of each?
- Risk evaluation: what could go wrong with each option? What is the worst case? What is reversible?
- Decision selection: which option best balances effectiveness, risk, and resource cost given the current constraints?
- Decision explanation: produce a human-readable explanation of why this decision was made — essential for trust and debugging

The Input/Output Contract

The reasoning engine has a strict input/output contract. Input is always a structured situation summary from the world model. Output is always a structured decision object: chosen action, confidence, justification, alternative options considered, conditions that would change this decision, and a request for any additional world model information needed. The reasoning engine never writes directly to memory or the knowledge graph — it produces output and the execution layer acts on it.

LLM Integration

Use the LLM in a structured way — not as a free-form chat interface, but as a reasoning API. The system prompt defines the reasoning engine's role, constraints, and output format. The user message contains the structured situation summary. The response must be a JSON object matching the decision schema. Parse and validate the response before passing it to the planner.

Use chain-of-thought prompting: instruct the LLM to reason step by step before producing its structured output. This makes the reasoning more reliable and makes the justification field genuinely informative. Validate that the output JSON is valid and complete — if not, retry with an error message that explains what was missing. Most validation failures resolve on the first retry.

The Rule-Based Layer

Not every decision needs an LLM. Build a rule engine that runs before the LLM and handles high-confidence, low-ambiguity situations: if queue depth exceeds 95% capacity, immediately reduce input rate. If three consecutive health checks fail, mark service as down. These rules are fast, free, explainable, and correct for the cases they cover. The LLM handles the cases the rules cannot.

Technology Stack for Phase 8

Component	Technology
Primary LLM	Any capable model via API: Claude, GPT-4, or a locally hosted open model (Mistral, LLaMA) for lower latency
Structured output	LLM API function calling / tool use for guaranteed JSON output structure
Rule engine	Python with a simple condition-action rule store in PostgreSQL
Prompt management	LangChain for prompt templating and chain management
Decision store	PostgreSQL: every decision is logged with its input situation, output, and eventual outcome

Phase 9

Build the Planning System

Converting decisions into executable sequences — goals decomposed into concrete actions

The reasoning engine decides what to do. The planning system figures out exactly how to do it: the sequence of steps, the dependencies between them, the conditions for proceeding at each step, and the rollback strategy if something goes wrong. Planning is what makes the system robust rather than brittle.

The Planning Hierarchy

Plans are hierarchical: a high-level goal decomposes into subgoals, each of which decomposes into tasks, each of which decomposes into atomic actions. This hierarchy is important because it allows partial execution (if a subgoal fails, you retry at the subgoal level, not from the beginning), progress tracking (you can report how far into a plan the system is), and learning (which subgoal sequences reliably succeed?).

Level	Example
Goal	Restore system stability
Subgoal 1	Relieve database pressure
Task 1.1	Add read replica
Action 1.1.1	Call scaling API with replica_count=3
Action 1.1.2	Wait for replica readiness (poll every 10s)
Action 1.1.3	Verify read distribution via metrics
Subgoal 2	Drain queue backlog
Task 2.1	Increase consumer concurrency
Action 2.1.1	Update consumer group configuration

Procedural Memory Integration

Before constructing a new plan from scratch, the planner consults procedural memory: does a procedure already exist for this type of situation? If so, the planner instantiates it (fills in the specific entity IDs and parameters) rather than generating a new plan. This makes the planner faster, more consistent, and capable of improving over time as successful plans are stored back into procedural memory.

Plan Validation

Every plan must be validated before execution: does it violate any constraints? Are all required permissions available? Are there conflicting plans already in execution? Does it have a rollback path for every destructive step? The validator runs synchronously before the plan reaches the execution layer. A plan that fails validation is sent back to the reasoning engine with an explanation.

Technology: LangGraph

Represent plans as directed graphs using LangGraph. Each node is a task or action. Each edge is a dependency or conditional transition. This representation handles branching logic (if step 3 fails, go to rollback path), parallel execution (steps 4 and 5 can run simultaneously), and progress tracking (which nodes are complete, in-progress, or blocked) naturally.

Phase 10

Execution Layer and the Feedback Loop

Actions meet reality — and the system learns from what happens

The execution layer is where the plan becomes real. It translates each action in the plan graph into a concrete operation: an API call, a file write, a subprocess spawn, a message to another agent. Crucially, every action the execution layer takes is an event — it flows back through the perception layer, through memory, through the world model, and back to the reasoning engine. This is the feedback loop, and it is what makes the system adaptive.

Execution Layer Design

The execution layer has four responsibilities:

12. Action dispatch: call the appropriate tool, API, or system to execute the action
13. Result observation: capture the result of the action and normalize it into the unified event schema
14. State confirmation: verify that the world state actually changed as expected — not just that the API call returned 200
15. Plan update: mark the completed step in the plan graph and trigger the next step or rollback path

The Confirmation Step Is Critical

An API call returning success does not mean the action achieved its goal. The execution layer must verify the expected environmental change: after scaling the database read replica, it should query the metrics endpoint and confirm that read load distribution has actually shifted. If the confirmation fails, the plan graph takes the failure path — do not proceed as if success occurred.

The Feedback Loop

The feedback loop is what distinguishes a cognitive architecture from an automation pipeline. Here is exactly how it works:

16. The execution layer completes an action and emits an `action_completed` event with the observed result
17. The perception layer normalizes this event and publishes it to the event bus
18. The memory router stores it in episodic memory and updates the knowledge graph
19. The world model incorporates the update: the entity's state changes, the situation changes
20. The situation assessor re-evaluates: is the goal achieved? Is the situation better, worse, or unchanged?

21. If the goal is not yet achieved, the reasoning engine receives the updated situation and produces the next decision
22. Over time, outcomes are matched to decisions: when the same type of situation arises again, the reasoning engine knows which decisions previously worked

Learning from the Feedback Loop

The feedback loop enables learning in two forms:

Short-term adaptation: within a single incident, if an action fails, the system tries a different approach without human intervention. This is reactive learning.

Long-term learning: after an incident is resolved, the system analyzes the decision sequence that led to resolution and stores it in procedural memory. Next time a similar situation arises, the planner will find this procedure and use it. This is experiential learning. It is not gradient descent or neural network training — it is structured experience accumulation, which is both more interpretable and more appropriate for a planning system.

Observability Layer

The feedback loop only produces learning if every decision is traceable. Build the observability layer as part of Phase 10:

- Every decision is logged: input situation, chosen action, confidence, alternatives considered
- Every action is logged: plan step ID, what was called, what was returned, confirmation result
- Every outcome is correlated with the decision that produced it
- Dashboards show: what is the system currently doing, what is it planning to do, what did it do in the last hour, and what was the outcome

This observability infrastructure is not just for debugging. It is the foundation of the learning system — the dataset that future model improvements will train on.

Phase 11

Multi-Agent Coordination

Scaling cognition across specialized agents — a distributed cognitive organism

Once the single-agent loop is stable and the feedback loop is producing reliable learning, you can scale to multiple agents. Multi-agent systems are not just "more agents doing the same thing" — they represent a qualitative change: specialization, parallel execution, delegation, and emergent behaviors that no single agent could produce alone.

The Agent Specialization Model

Different agents are good at different things. Specialization improves performance because each agent can be optimized for its role rather than being a generalist that does everything poorly. The cognitive architecture naturally suggests these agent roles:

Agent Type	Responsibilities
Perception Agent	Dedicated to managing adapters, validating events, and maintaining the event bus — owns the perception layer
Memory Agent	Manages the routing, consistency, and health of all four memory stores
World Model Agent	Maintains the live world model, runs the situation assessor, serves queries to other agents
Planner Agent	Translates decisions into validated plan graphs, consults procedural memory, manages plan state
Executor Agent	Dispatches actions, confirms outcomes, handles retries, reports results back to the bus
Monitor Agent	Watches the behavior of all other agents, detects when they are failing or misbehaving, escalates
Learning Agent	Analyzes completed incidents, extracts lessons, updates procedural memory, tracks model performance

The Shared Architecture

All agents share the same event bus, the same memory stores, and the same world model. No agent writes directly to another agent's state — all communication is through the event bus. This decoupling is what makes the system robust: any agent can fail and restart without corrupting the state of the others, because the state lives in shared infrastructure rather than in the agent process.

The Perception Bus Contract

In a multi-agent system, the event bus is the single source of truth. Every agent reads from it; every agent writes normalized events back to it. No agent should write directly to the knowledge graph, to episodic memory, or to working memory — all writes go through the memory router, which processes events from the bus. This prevents race conditions, ensures all agents have a consistent view of the world, and means every state change is automatically logged.

Coordination Patterns

Agents coordinate through events, not through direct function calls. When the world model agent detects a critical situation, it publishes a `situation_critical` event. The planner agent subscribes to this event and begins planning. The executor agent subscribes to `plan_ready` events. The monitor agent subscribes to all events and tracks whether the other agents are responding within expected time windows. If the executor agent stops responding, the monitor agent publishes an `agent_unresponsive` event, which triggers the planner to reroute the execution to an alternative executor.

Phase 12

Predictive Dynamic Models

When the system has enough data — learn the dynamics of the environment itself

This phase should not be started until Phase 11 is stable and the system has been running in production for at least several weeks. The reason is simple: the predictive models in this phase require training data — specifically, a rich history of events, decisions, outcomes, and world model states. That history only exists after extended operation.

What Predictive Modeling Adds

Until this phase, all predictions in the system are based on statistical trends and explicit rules: "at this rate of growth, the queue will reach capacity in four minutes." These predictions are useful but limited — they only extrapolate existing trends linearly and cannot model the non-linear dynamics of complex systems.

Graph Neural Networks (GNNs) change this. Because the world model is structured as a graph — entities as nodes, relationships as edges — GNNs can learn the dynamics of how failures propagate through the graph. They can answer questions like: given the current state of these twenty entities and their relationships, which ones are most likely to fail in the next thirty minutes? No rule-based system can answer this question reliably. A well-trained GNN can.

What to Build

- Failure propagation model: given a failure at node X, which downstream nodes are most likely to be affected, and in what order? Train on historical incident data from episodic memory.
- Anomaly detection model: learn the normal state distribution for each entity and relationship, and flag deviations that exceed learned thresholds.
- Recovery time prediction: given a specific type of failure, how long does recovery typically take? This helps the planning system set realistic timeouts.

Start Simple, Graduate to GNNs

Before building a GNN, implement the same predictions as gradient boosting models trained on tabular features extracted from the world model. These are much easier to train, debug, and explain. Only move to GNNs when the gradient boosting models are clearly limited by their inability to model graph structure. In most real systems, well-tuned gradient boosting outperforms GNNs on small data.

Your First Prototype: Scope and Success Criteria

Do not build all twelve phases as a prototype. Build phases 2 through 7 targeting a single, well-defined scenario. Here is exactly what to build and why.

Target Scenario

Monitor a set of services with known dependency relationships. When one goes into a degraded state, identify the root cause by traversing the dependency graph, generate a situation summary, and produce a human-readable recommended action. Do not automatically execute the action — produce a recommendation that a human can review and approve.

This scenario is deliberately small. It exercises every cognitive layer — perception, memory, knowledge graph, temporal tracking, world model, and reasoning — without requiring the execution loop or multi-agent coordination. It is completable in two to four weeks and produces something genuinely demonstrable.

What the Prototype Demonstrates

Capability	How It Is Demonstrated
Perception	Live ingestion of simulated service metrics and logs through the event bus
Memory routing	Events correctly stored in episodic memory and used to update the knowledge graph
Knowledge graph	Service dependency relationships correctly represented and queryable
Temporal awareness	Trend detection showing service degradation developing over time, not just a single event
World model	A structured query response that shows current state, causal chain, and affected entities
Reasoning	A natural language recommendation that correctly identifies the root cause and proposes the right action

The Extensibility Proof

After the prototype works for infrastructure monitoring, prove its domain-agnosticism by extending it to a completely different domain with minimal architectural changes.

Options:

- User behavior analysis: swap the infrastructure adapters for user event adapters. The same memory, knowledge graph, world model, and reasoning layer now tracks user journeys, identifies friction points, and recommends UX improvements.

- Research assistant: add a paper-ingestion adapter that reads abstracts from arXiv. The knowledge graph now tracks concepts and citations. The reasoning engine surfaces related work when you add a new concept.
- Personal productivity: add calendar, email, and task adapters. The system now tracks your commitments, identifies conflicts, and recommends priority adjustments.

Each of these requires new perception adapters and domain-specific knowledge graph schemas. The core architecture — event bus, memory router, world model, reasoning engine — requires no changes. This extensibility proof is the clearest demonstration that you have built a cognitive architecture, not a monitoring tool.

Complete Technology Stack Reference

Layer	Technology	Purpose
Event schema	Python + Pydantic v2	Validated, typed event envelope used by all layers
Event bus	Apache Kafka / Redpanda	Persistent, replayable ordered event stream
Perception adapters	Python + asyncio	Concurrent ingestion from multiple signal sources
Working memory	Redis + Redis Streams	Fast in-memory state with automatic TTL
Episodic memory (time)	PostgreSQL + TimescaleDB	Time-series storage for temporal queries
Episodic memory (semantic)	Qdrant	Vector similarity for "find past situations like this"
Semantic memory	Neo4j	Property graph for entity relationships and causal chains
Procedural memory	PostgreSQL	Structured procedure store with success rate tracking
Memory router	Python Kafka consumer	Routes events to correct memory stores
Temporal analysis	Apache Flink / Faust	Streaming aggregations and trend detection
World model API	FastAPI	Structured query interface consumed by reasoning layer
Situation assessor	Python + rules + small ML	Compresses world state into reasoning-ready summary
Reasoning engine	LLM API + LangChain	Structured decision-making over situation summaries
Rule engine	Python + PostgreSQL	Fast deterministic rules for high-confidence situations
Planner	LangGraph	Directed graph representation of executable plans
Executor	Python + Celery	Async action dispatch with retry and confirmation
Observability	Prometheus + Grafana	Metrics, traces, and decision dashboards
Testing	pytest + synthetic events	Validate each layer independently against schema

The Principles That Will Keep You on Track

1. The Schema Is a Contract, Not a Suggestion

Every component that produces events must validate them against the schema before publishing. Every component that consumes events must handle invalid events gracefully. Do not introduce exceptions "just for this one source." Exceptions compound into architectural rot.

2. The Feedback Loop Is Not Optional

A system that acts but does not observe the results of its actions is not a cognitive architecture — it is a pipeline. If the feedback loop is not working, the entire system is just a slightly smarter automation script. Prioritize building and verifying the feedback loop early.

3. LLMs Are Components, Not Architecture

Resist the temptation to make the LLM do everything: perceive, remember, reason, plan, and execute in a single prompt. This approach fails at scale, is untraceable, is unreliable, and is not a cognitive architecture. Keep the LLM in the reasoning layer where it belongs.

4. Observability Is Not a Feature — It Is Infrastructure

Every decision the system makes must be traceable: what was the input situation, what options were considered, what was chosen, and what was the outcome. Without this trace, you cannot improve the system, debug failures, or trust its decisions. Build observability alongside each layer, not after.

5. One Domain, Then Many

Master the architecture with one domain before extending it. The architecture is domain-agnostic, but your first implementation will reveal many integration challenges that are much easier to solve in a familiar domain. Once the single-domain system is stable, extending to a new domain should require only new adapters and schema extensions — not architectural changes.

6. Simple Before Complex

At every layer, start with the simplest working implementation. Simple trend extrapolation before GNNs. Rule-based routing before ML routing. Direct Cypher queries before graph neural network embeddings. The simple implementation will tell you

whether the layer is working correctly, which is necessary before adding complexity. Premature complexity produces systems that look impressive and fail mysteriously.

You are not building a smarter chatbot.

You are building a persistent cognitive substrate —

a structure for intelligence that can be applied to any environment, any domain, any scale.